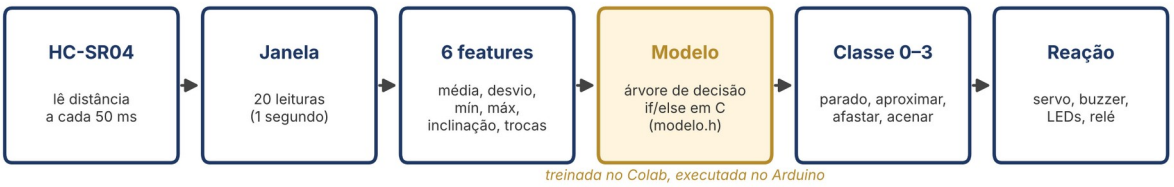
MINICURSO DE ROBÓTICA

Robô Sentinela com Machine Learning embarcado

Arduino Uno · HC-SR04 · servo · árvore de decisão treinada em Python e executada em C



Carga horária: 4 horas | Nível: intermediário | Formato: prático, em grupos

Público: primeiros períodos de Engenharia de Computação

4 grupos (kit + servo cada) + kit do professor para demonstração

Prof. Paulo César da Silva Junior

Roteiro do Professor + Apostila do Aluno

1. O que vamos construir

Um robô sentinela: uma "cabeça" com sensor ultrassônico montada sobre um servo motor, que patrulha o ambiente, percebe quando alguém se aproxima e reconhece gestos feitos com a mão. O reconhecimento não é feito com regras escritas à mão — é feito por um modelo de Machine Learning que os próprios alunos treinam com dados gravados no seu robô e depois embarcam no Arduino.

Gesto	Classe	Reação do robô
Mão parada / ninguém	0 PARADO	Continua vigiando; sem ninguém por 3 s, volta a patrulhar
Mão se aproximando	1 APROXIMAR	ALARME: sirene + relé ("holofote")
Mão se afastando	2 AFASTAR	"Tchau": bip descendente, volta a patrulhar
Aceno (vai-e-vem)	3 ACENAR	AMIGO: cabeça balança + melodia

Por que este curso é "intermediário"

- **Arquitetura de robô:** percepção → decisão → ação, organizada como máquina de estados.
- **Programação de tempo real simples:** loop sem delay() usando millis(), amostragem em ritmo fixo.
- **Pipeline completo de ML:** coleta e rotulagem de dados, extração de features, treino/teste, matriz de confusão, overfitting e generalização.
- **ML embarcado (TinyML):** o modelo treinado em Python é convertido automaticamente em código C e roda em um microcontrolador de 2 KB de RAM.

Objetivos de aprendizagem

Ao final, o aluno deve ser capaz de:

- Montar e testar um sistema com sensores e atuadores, identificando falhas de montagem.
- Implementar uma máquina de estados não bloqueante no Arduino.
- Explicar a diferença entre programar regras e aprender regras a partir de dados.
- Coletar um dataset rotulado, treinar e avaliar um classificador, e reconhecer overfitting.
- Integrar um modelo treinado ao firmware e validar o comportamento no robô real.

2. Organização e materiais

Turma

- 4 grupos de 3 a 5 alunos, cada um com 1 kit + 1 servo. O 5º kit fica com o professor, já montado e com o robô final funcionando, para a demonstração de abertura e para substituir peças.
- Cada grupo precisa de 1 computador com a Arduino IDE. Para o treino do modelo, ao menos 1 computador por grupo com navegador e conta Google (Google Colab). Plano B: o professor treina no próprio computador com os arquivos de dados dos grupos.

Por grupo

Item	Qtd.	Uso no robô
Placa UNO R3 + cabo USB (kit)	1	Cérebro
Protoboard 400 pontos (kit)	1	Display, LEDs, resistores
Sensor ultrassônico HC-SR04 (kit)	1	"Olho" — mede distância
Micro servo SG90	1	"Pescoço" — gira a cabeça
Display 7 segmentos 1 dígito (kit)	1	Mostra estado (P, A) e classe (0–3)
LEDs vermelho, verde e azul (kit)	1 de cada	Alerta, patrulha, "ouvindo"
Buzzer 5V (kit)	1	Bips, sirene, melodia
Módulo relé 1 canal (kit)	1	"Holofote" / sirene externa
Push button (kit)	1	Botão GRAVAR na coleta de dados
Sensor LDR (kit)	1	Detecta escuro → liga o relé
Resistores 620 Ω (kit)	7	Segmentos do display
Resistores 560 Ω (kit)	3	LEDs
Resistor 1 kΩ (kit)	1	Divisor de tensão do LDR
Capacitor eletrolítico 100 µF (kit)	1	Estabiliza a alimentação do servo
Jumpers macho-macho (kit)	~25	Ligações na protoboard
Jumpers macho-fêmea	4	HC-SR04 na cabeça (comprar à parte)
Fita dupla face / abraçadeira / papelão	—	Fixar o HC-SR04 no braço do servo e o servo na mesa

> **Atenção: itens que o kit não traz**
>
> 1) Jumpers macho-fêmea: o HC-SR04 tem pinos machos e vai ficar em cima do servo, longe da protoboard. Os jumpers do kit são só macho-macho. Compre 4 macho-fêmea por grupo (são baratos).
>
> 2) Servo SG90: o curso conta com os 4 servos já disponíveis. O conector fêmea do servo aceita os jumpers macho-macho do kit.

Checklist do professor (antes da aula)

- Instalar a Arduino IDE (2.x) em todos os computadores e testar o upload (driver CH340 para placas compatíveis).
- Rodar o sketch 01 em cada kit. Anotar: o display é cátodo ou ânodo comum? O buzzer é ativo ou passivo? O relé liga com LOW ou HIGH? (veja a seção 3).

- Montar a cabeça de cada robô: HC-SR04 colado no braço (horn) do servo com fita dupla face; servo preso a uma base (caixinha, papelão grosso) para não sair andando.
- Deixar o kit do professor montado com o sketch 04 e um modelo treinado com os seus próprios gestos (faça a coleta do sketch 03 antes).
- Distribuir a pasta de códigos (Codigos_Robo_Sentinela.zip) e abrir o notebook no Colab uma vez para conferir que roda (use dados_exemplo_simulado.csv).

3. Montagem: pinagem completa

A mesma pinagem vale para os 4 sketches — a montagem é feita uma única vez, na Parte 1.

Componente	Pino Arduino	Ligação
Botão GRAVAR	D2	Outro terminal no GND (usa pull-up interno)
Display seg. a, b, c, d	D3, D4, D5, D6	Cada um via resistor 620 Ω
Display seg. e, f, g	A3, A4, A5	Pinos analógicos usados como digitais; via 620 Ω
Display pino comum	GND ou 5V	GND = cátodo comum; 5V = ânodo comum
HC-SR04 TRIG / ECHO	D7 / D8	VCC no 5V, GND no GND
Servo (fio laranja/amarelo)	D9	Vermelho no 5V, marrom no GND
Capacitor 100 µF	—	Entre 5V (+) e GND (−), perto do servo
Buzzer	D10	Negativo no GND
LED vermelho / verde / azul	D11 / D12 / D13	Anodo no pino; catodo via 560 Ω ao GND
LDR	A0	LDR entre 5V e A0; resistor 1 kΩ entre A0 e GND
Módulo relé (IN)	A2	VCC no 5V, GND no GND

Diagrama de montagem

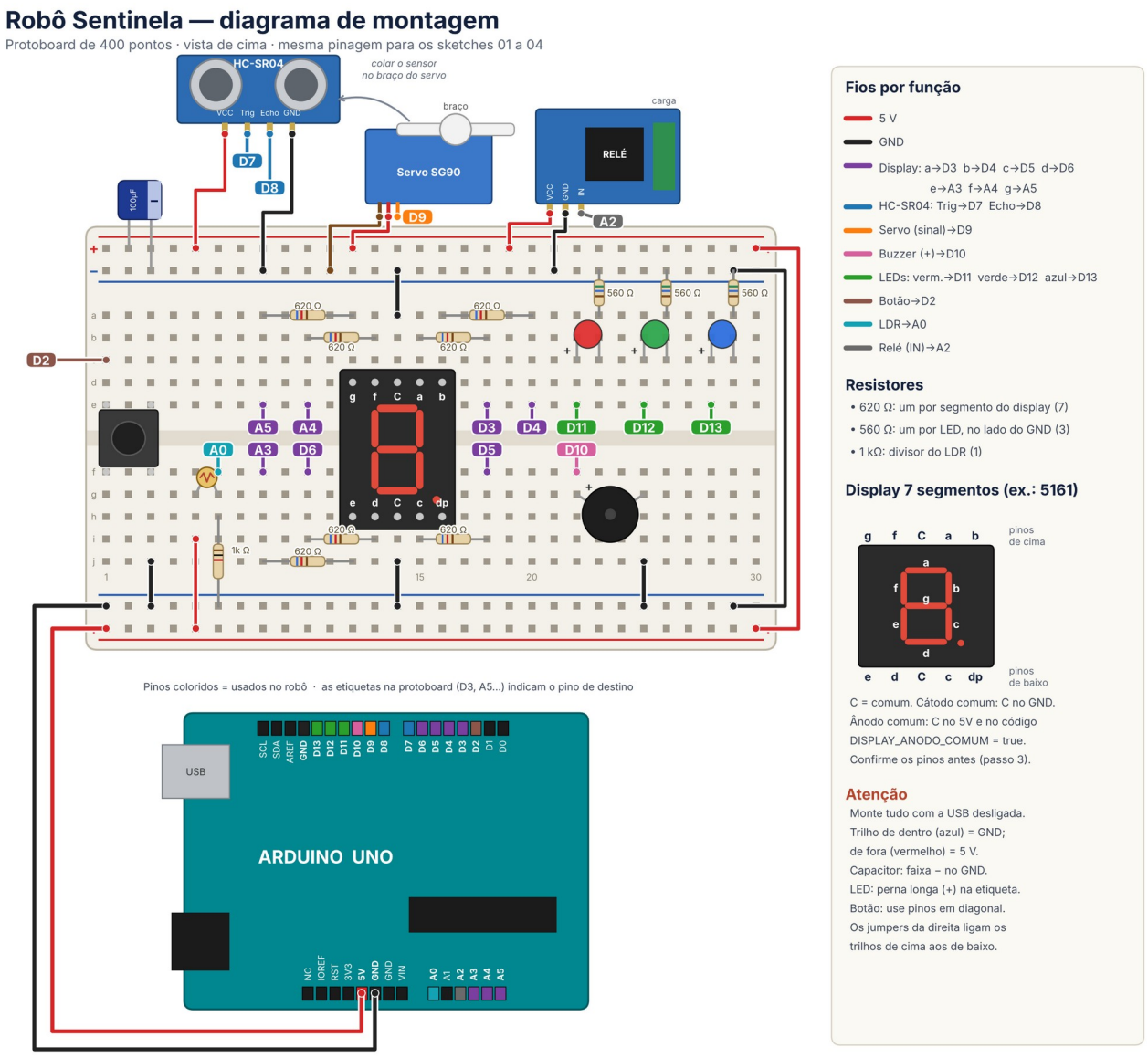


Figura — Montagem na protoboard de 400 pontos. Cada etiqueta (D3, A5…) indica o pino do Arduino de destino daquele fio.

Por que estes pinos? (vale explicar em sala)

A biblioteca Servo usa o Timer1 do ATmega328P — isso desliga o PWM dos pinos 9 e 10. A função tone() usa o Timer2 — afeta o PWM dos pinos 3 e 11. Como nada no robô usa analogWrite(), podemos colocar servo e buzzer nesses pinos sem conflito.

O Uno tem 14 pinos digitais, mas D0/D1 são da serial (USB). Faltaram pinos para o display, então A3–A5 entram como saídas digitais comuns — todo pino analógico do Uno também funciona como digital.

Ajustes no início de cada sketch

```
const bool DISPLAY_ANODO_COMUM = false; // true se o comum do display vai no 5V
const bool BUZZER_ATIVO        = false; // true se o buzzer apita só com 5V
const bool RELE_ATIVO_BAIXO    = true;  // a maioria dos módulos relé liga com LOW
```

Dicas de montagem na protoboard de 400 pontos

Display no centro da protoboard, com os 7 resistores saindo dele. LEDs na ponta. HC-SR04, servo e relé ficam fora da protoboard, ligados por jumpers.

Se o Arduino reiniciar quando o servo se mexe, é queda de tensão no USB: confira o capacitor de 100 µF (polaridade!) e use uma porta USB do próprio computador, não de hub.

4. Cronograma (4 h)

Horário	Bloco	Atividade	Sketch
0:00–1:40	Parte 1	Arduino passo a passo: montar e testar cada componente	etapas 1–9 + 01
1:40–1:50	—	Intervalo	—
1:50–2:20	Bloco 2	Comportamento por regras: máquina de estados com millis()	02
2:20–3:15	Bloco 3	Machine Learning: coleta de dados, treino no Colab, avaliação	03 + Colab
3:15–3:50	Bloco 4	Integração do modelo no robô, testes e ajustes	04
3:50–4:00	Encerramento	Teste cruzado entre grupos	04

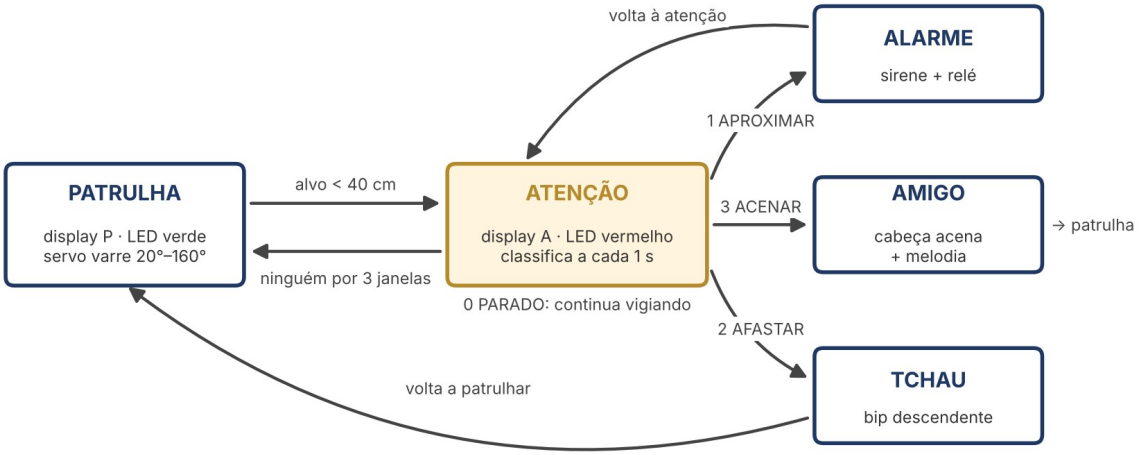


Figura 1 — Máquina de estados do robô final (sketch 04).

5. Parte 1 — Arduino passo a passo (1h40)

A Parte 1 tem apresentação própria ("Minicurso Parte 1 — Arduino passo a passo") e funciona como o Bloco 1: em vez de montar tudo de uma vez, cada componente é montado direto no seu lugar definitivo e testado com um código curto antes do próximo. Ao final, a protoboard está igual ao diagrama da seção 3.

Etapa	Componente	Código	Tempo
—	Abertura, Arduino, IDE, protoboard, kit, resistores	—	15 min
0	Energia nos trilhos (5 V e GND)	—	3 min
1	LED vermelho (D11)	etapa1_led	8 min
2	LEDs verde e azul (D12, D13)	etapa2_leds	6 min
3	Botão (D2, pull-up interno)	etapa3_botao	8 min
4	Buzzer (D10)	etapa4_buzzer	6 min
5	LDR + 1 kΩ (A0) — calibrar o limiar	etapa5_ldr	8 min
6	Display 7 segmentos + contador	etapa6_display, etapa6b_contador	20 min
7	HC-SR04 (D7, D8)	etapa7_ultrassom	8 min
8	Servo (D9) + capacitor 100 µF	etapa8_servo	10 min
9	Relé (A2)	etapa9_rele	5 min
✓	Teste geral do robô montado	01_teste_hardware	5 min

> ✔️ **Checkpoint da Parte 1**
> Todos os componentes funcionaram no 01_teste_hardware e o grupo anotou: tipo de display, tipo de buzzer, lógica do relé e valor do LDR no escuro.

6. Bloco 2 — Comportamento por regras (30 min)

Objetivo

Dar comportamento ao robô com uma máquina de estados e mostrar, na prática, o limite de regras escritas à mão.

Roteiro

1. (8 min) Conceito: máquina de estados (PATRULHA, ATENCAO, ALARME) e loop não bloqueante com millis().
2. (7 min) Upload do sketch 02. Observar a patrulha, a detecção (alvo < 40 cm) e o alarme (alvo < 12 cm). No escuro, o relé acende na atenção.
3. (7 min) Ajustes: mudar LIMIAR_CM, PERIGO_CM, velocidade da varredura (PASSO_MS, ANG_PASSO). Discutir por que exigimos 2 leituras seguidas para disparar.
4. (8 min) Desafio: escrever uma regra que diferencie APROXIMAR, AFASTAR e ACENAR. Deixe tentar — a maioria vai perceber que os limiares dependem da pessoa, da velocidade e da distância.

Conceito-chave: o padrão de loop sem delay()

```
void loop() {
  unsigned long agora = millis();
  if (agora - tUltimoPasso < PASSO_MS) return;  // ainda não é hora
  tUltimoPasso = agora;

  float d = lerDistanciaCm();
  switch (estado) {
    case PATRULHA: /* move a cabeça, procura alvo */  break;
    case ATENCAO:  /* vigia, decide alarme/voltar */  break;
    case ALARME:   /* sirene por 3 s */               break;
  }
}
```

> **Ponte para o Bloco 3**
> Em vez de nós inventarmos os limiares ("se a inclinação for menor que X e o desvio maior que Y..."), vamos mostrar exemplos ao computador e deixar um algoritmo encontrar as regras. Isso é aprendizado supervisionado.

7. Bloco 3 — Machine Learning (55 min)

Objetivo

Percorrer o ciclo completo de um projeto de ML: dados → features → treino → avaliação → exportação.

7.1 Conceitos (15 min)

- **Dataset rotulado:** cada exemplo é um gesto gravado + a resposta certa (a classe).
- **Janela:** 20 leituras de distância, uma a cada 50 ms (1 segundo de movimento).
- **Features:** 20 números crus são difíceis de comparar; resumimos a janela em 6 características com significado físico.
- **Treino × teste:** o modelo aprende com 75% dos dados e é avaliado nos 25% que nunca viu.
- **Árvore de decisão:** uma sequência de perguntas do tipo "inclinação ≤ −19?". Vira if/else em C — cabe nos 2 KB de RAM do Uno.

Feature	O que mede	Ajuda a reconhecer
media	Distância média na janela (cm)	Se há alguém perto ou não
desvio	Quanto a distância variou (desvio padrão)	Mão parada × mão em movimento
minimo / maximo	Menor e maior distância	Alcance do gesto
inclinacao	Tendência da reta (cm/s): negativa = aproximando	APROXIMAR × AFASTAR
trocas	Quantas vezes a mão inverteu o sentido	ACENAR (vai-e-vem)

7.2 Coleta de dados (15 min)

1. Upload do sketch 03. Abrir o Serial Monitor (9600 baud).
2. Digitar a classe (0, 1, 2 ou 3) e Enter — o display mostra a classe escolhida.
3. Apertar GRAVAR: bip grave = prepare-se; bip agudo + LED azul = gravando por 1 s. Fazer o gesto durante o LED azul.
4. Repetir até ~20 gravações por classe (o Serial Monitor mostra a contagem). Variar: quem faz o gesto, a velocidade, a distância.
5. Copiar todo o texto do Serial Monitor (Ctrl+A, Ctrl+C), colar num editor e salvar como dados_grupoX.csv.

Classe	Como fazer o gesto
0 PARADO	Metade das gravações com a mão parada a 10–30 cm; metade sem nada na frente
1 APROXIMAR	Mão começa a ~40 cm e chega a ~10 cm durante o LED azul
2 AFASTAR	Mão começa a ~10 cm e vai até ~40 cm
3 ACENAR	Vai-e-vem rápido da mão, de frente para o sensor, entre ~10 e ~30 cm

> **Erro mais comum**
> Gravar todas as amostras do mesmo jeito, com a mesma pessoa. O modelo decora aquele jeito e falha com outra pessoa. Diversidade nos dados vale mais do que quantidade.

7.3 Treino no Colab (20 min)

1. Abrir colab.research.google.com → Arquivo → Fazer upload de notebook → treino_modelo_colab.ipynb.
2. Executar as células em ordem (Shift+Enter). Na célula 1, enviar o dados_grupoX.csv.
3. Observar os gráficos: qual feature separa melhor cada classe?
4. Treinar com PROFUNDIDADE = 4, ler a acurácia de treino e de teste e a matriz de confusão.
5. Rodar o experimento de profundidade (1 a 10) e discutir o gráfico de overfitting.
6. Na última célula, o notebook gera e baixa o arquivo modelo.h.

Exemplo de modelo.h gerado

```
int classificar(const float *f) {
  if (f[4] <= -18.9750f) {          // inclinacao
    return 1;                        // APROXIMAR
  } else {
    if (f[1] <= 2.4650f) {           // desvio
      return 0;                      // PARADO
    } else {
      if (f[5] <= 3.5000f) return 2; // trocas -> AFASTAR
      else                 return 3; // ACENAR
    }
  }
}
```

7.4 Discussão (5 min)

- Treino 100% e teste 70%: o que aconteceu? (overfitting — a árvore decorou).
- Onde a matriz de confusão mostra mais erros? Faz sentido fisicamente? (ex.: AFASTAR lento parece PARADO).
- Compare a árvore com as regras que vocês tentaram no Bloco 2.

> **Plano B (sem internet ou sem Colab)**
> Os grupos entregam o .csv ao professor (pendrive), que roda o mesmo notebook no próprio computador (Jupyter) e devolve o modelo.h. Se a coleta falhar, o arquivo dados_exemplo_simulado.csv permite rodar o notebook inteiro — mas é simulado e não substitui os dados reais do sensor.

8. Bloco 4 — Integração: o robô ganha cérebro (35 min)

Roteiro

1. (5 min) Copiar o modelo.h baixado para a pasta 04_sentinela_ml, substituindo o de exemplo.
2. (5 min) Conferir que os ajustes do topo do sketch (display, buzzer, relé, LDR_ESCURO) estão iguais aos do sketch 01.
3. (15 min) Upload e testes. O Serial Monitor mostra as features e a classe de cada janela — use para entender os erros.
4. (10 min) Se o robô errar um gesto com frequência: gravar mais exemplos dessa classe (sketch 03), retreinar e subir de novo. Esse é o ciclo real de quem trabalha com ML.

O coração do sketch 04

```
janela[nJanela++] = d;                  // 1 leitura a cada 50 ms
if (nJanela < N_AMOSTRAS) return;       // janela ainda incompleta

calcularFeatures(janela, N_AMOSTRAS, f); // mesmas 6 features da coleta
int classe = classificar(f);             // <- o modelo treinado no Colab
mostrar(DIGITOS[classe]);

if      (classe == 1) reagirAlarme();
else if (classe == 2) { reagirTchau(); irPara(PATRULHA); }
else if (classe == 3) { reagirAmigo(); irPara(PATRULHA); }
```

> **Por que as features precisam ser idênticas**
>
> O modelo aprendeu a partir dos números calculados pela função calcularFeatures() do sketch 03. Se o sketch 04 calcular de outro jeito (outra janela, outro período, outro filtro), o modelo recebe números que nunca viu e erra. Por isso a função é uma cópia exata — e N_AMOSTRAS e PERIODO_MS também.

9. Encerramento: teste cruzado (10 min)

- Cada grupo apresenta o robô em 2 minutos: a acurácia no Colab e uma demonstração.
- Depois, um aluno de outro grupo faz os gestos no robô. Ele acerta tanto quanto com o próprio grupo?
- Fechamento: os dados de treino definem o que o modelo sabe. Para generalizar, é preciso dados de muitas pessoas — e foi isso que a célula 7 do notebook (treinar com todos os grupos menos um) mostrou.
- Gancho para o curso: sistemas embarcados, inteligência artificial, processamento de sinais, controle.

10. Solução de problemas

Sintoma	Causa provável	O que fazer
Distância sempre 200	TRIG/ECHO invertidos ou sem 5V	Conferir D7 = TRIG, D8 = ECHO e alimentação
Distância oscila muito	Superfície inclinada ou macia; mão muito longe	Testar com objeto plano a 10–40 cm
Arduino reinicia quando o servo mexe	Queda de tensão no USB	Capacitor 100 µF no 5V/GND; porta USB direta
Servo treme parado	Ruído de alimentação	Mesmo acima; prender bem o servo na base
Display mostra tudo invertido	Tipo de display trocado	Inverter DISPLAY_ANODO_COMUM
Buzzer sempre com o mesmo som	Buzzer ativo	BUZZER_ATIVO = true (melodias viram bips)
Relé ligado o tempo todo	Módulo com lógica invertida	Inverter RELE_ATIVO_BAIXO
Erro 'modelo.h: No such file'	Arquivo fora da pasta do sketch	modelo.h deve estar ao lado do .ino
Colab: '0 amostras carregadas'	Arquivo copiado sem as linhas de dados	Abrir o .csv e conferir as linhas com 7 números
Robô confunde dois gestos	Poucos ou pouco variados exemplos	Gravar mais dessa classe e retreinar
Robô reage a qualquer coisa como APROXIMAR	A pessoa chega para fazer o gesto	Esperar o bip da ATENÇÃO antes de gesticular

11. Desafios extras (para quem terminar antes)

- **Nova classe:** "dois acenos" ou "mão parada muito perto". Grave, retreine e programe uma reação nova.
- **Modelo da turma:** envie os CSVs dos 4 grupos juntos ao Colab e compare com o modelo de um grupo só.
- **Outro algoritmo:** no notebook, compare a árvore com k-NN (KNeighborsClassifier). Por que o k-NN seria difícil de levar para o Uno?
- **Radar:** no sketch 02, imprima ângulo e distância e desenhe um mapa da sala no Python.
- **Temperatura:** o kit tem um sensor NTC — use-o em A1 para um alarme de temperatura alta.

Anexo A — Arquivos do curso

Pasta / arquivo	Para que serve	Quando
parte1-arduino-passo-a-passo/etapa1_led … etapa9_rele	Um código de teste por componente	Parte 1
parte2-robo-sentinela/01_teste_hardware	Testa o robô inteiro; Serial Plotter	Fim da Parte 1
parte2-robo-sentinela/02_sentinela_regras	Robô por regras (máquina de estados)	Bloco 2
parte2-robo-sentinela/03_coleta_dados	Grava gestos rotulados em CSV	Bloco 3
parte2-robo-sentinela/ml/ treino_modelo_colab.ipynb	Treina, avalia e exporta o modelo.h	Bloco 3
parte2-robo-sentinela/ml/ dados_exemplo_simulado.csv	Dados simulados (plano B)	Bloco 3
parte2-robo-sentinela/04_sentinela_ml	Robô final; modelo.h de exemplo para substituir	Bloco 4
docs/diagramas	Diagrama completo e diagramas de cada etapa	Montagem

Observação: a Arduino IDE exige que cada .ino fique numa pasta com o mesmo nome — por isso cada código tem a sua pasta.

Anexo B — Ficha de coleta do grupo

Grupo: _____ Integrantes: _____

Classe	Quem gravou / como variou	Qtd.	Acurácia teste
0 PARADO			
1 APROXIMAR			
2 AFASTAR			
3 ACENAR			

Profundidade escolhida: _____ Acurácia treino: _____ % Acurácia teste: _____ %

Gesto que o robô mais errou e por quê: _____

_____

Acerto no teste cruzado (aluno de outro grupo): _____ de _____ gestos